

Kassidy Shomo

Professor Kraya

CS 499: Computer Science Capstone

June 18, 2026

Enhancement Two: Algorithms and Data Structure

This artifact was originally developed on January 8 as part of CS 405: Secure Coding. At the time, the program was designed to demonstrate numeric overflow and underflow behavior in C++ using template-based functions that simulate repeated addition and subtraction across multiple primitive data types. The original implementation relied on loop-based iteration to build results step-by-step, which made the behavior of overflow easier to observe, but also introduced unnecessary computational overhead from an algorithmic standpoint.

The reason I selected this artifact for my ePortfolio is because it clearly shows how algorithm design decisions affect both performance and structure of a solution. While the original version successfully demonstrated overflow and underflow conditions, it depended on repeated iteration in both `add_numbers` and `subtract_numbers`, which increased time complexity and made the solution less efficient than it needed to be for a computation that can be expressed mathematically in a single step.

In the earlier version of the artifact, the main issue was the reliance on iterative loops to reach a final value. Both functions repeatedly updated the result inside a loop, with overflow and underflow checks occurring during each iteration. While this approach is useful for

demonstrating how numeric limits are reached in real time, it is not necessary for calculating the final outcome when the relationship between the inputs is already known.

The enhanced version restructures this logic by replacing iteration with direct mathematical computation. Instead of building the result step-by-step, the updated functions calculate the final value using $\text{start} + (\text{increment} \times \text{steps})$ for addition and $\text{start} - (\text{decrement} \times \text{steps})$ for subtraction. This change reduces the time complexity from $O(n)$ to $O(1)$, since the computation now happens in a single operation rather than through repeated looping.

To make the impact of this change clearer, the differences between the original and enhanced implementation can be summarized below:

Original	Enhanced
Loop-based accumulation in <code>add_numbers</code> and <code>subtract_numbers</code>	Direct computation using $\text{start} \pm (\text{value} \times \text{steps})$
$O(n)$ time complexity due to repeated iteration	$O(1)$ constant-time computation
Overflow checked during each loop iteration	Overflow checked before computation using <code>check_safe_multiplication</code>
Step-by-step value updates inside loops	Single-step deterministic calculation
Same logic applied to all types inside iteration	Type-aware handling using long double intermediate values
Behavior easier to observe step-by-step	More efficient and closer to real-world arithmetic usage

This change is not just about efficiency, it also shifts where safety checks need to happen in the program. Since the multiplication now happens all at once, the enhanced version introduces a dedicated helper function, `check_safe_multiplication`, which evaluates whether $(\text{value} * \text{steps})$ would exceed the limits of the data type before any arithmetic operation is performed. This is a

change in structure compared to the original version, where overflow was detected gradually during each loop iteration.

Another improvement in the enhanced implementation is how floating-point values are handled. In the original version, floating-point checks were performed inside the loop using `std::isfinite`. In the updated version, floating-point operations are handled more consistently by using long double as an intermediate type in both `add_numbers` and `subtract_numbers`, and then validating the result using `std::isfinite(result)` before casting back to the original type. This reduces the chance of precision-related issues or invalid numeric states.

Even with these improvements, there are still limitations in the updated design. One trade-off is that the original version made it easier to observe how overflow develops step-by-step, which is now lost since the computation happens in a single operation. Another limitation is that the safety checks rely on numeric limits and type casting, which may behave slightly differently depending on compiler implementation or platform-specific precision for floating-point types.

Overall, the enhanced version shifts the focus from demonstrating iteration-based overflow behavior to implementing a more efficient and structured arithmetic model. It still preserves the original intent of detecting overflow and underflow, but it does so in a way that better reflects how these calculations would be handled in a performance-focused environment.

This artifact aligns strongly with algorithms and data structures because it demonstrates how changing the structure of a computation directly impacts time complexity and control flow. It also shows how template-based design can be used to apply consistent logic across multiple data types while still accounting for differences in numeric behavior. The shift from iterative

processing to constant-time computation highlights how algorithm design choices affect both efficiency and reliability.

This enhancement aligns with Course Outcomes 3 and 4. It aligns with Outcome 3 because it involves designing and evaluating an improved computing solution that reduces time complexity while still managing trade-offs related to numeric safety and overflow prevention. It aligns with Outcome 4 because it applies structured programming practices, templates, and standard library tools such as `std::numeric_limits`, `std::type_traits`, and `std::cmath` to build a more efficient and reliable implementation across multiple data types.

Through this process, I learned that improving algorithm efficiency often requires restructuring how a problem is solved rather than just optimizing existing logic. One of the main challenges was making sure that removing iteration did not weaken overflow detection, especially since the original design depended on step-by-step validation. Another challenge was designing pre-check logic that works consistently across different numeric types without introducing incorrect assumptions. Overall, this reinforced how algorithm design decisions affect both performance and the reliability of a system under edge conditions.